

Skill-Centric Unified Job Search Aggregator System — Phased Development Plan

Methodology

The project follows an **iterative, module-driven methodology**, structured around the four architectural stages defined in `Architecture.md`, rather than one monolithic build. Each phase is developed, unit-tested, and validated independently before integration with the next. This mirrors the pipeline itself: nothing downstream is built against a stage that hasn't been validated upstream.

Phase 0 — Project Setup

Goal: A working local environment before any feature code is written.

Initialize repo structure (frontend, backend, worker, infra).

Docker Compose skeleton: Postgres, Redis, API, worker containers wired together.

CI pipeline skeleton (lint + test on push).

Environment/secrets management scaffolding (see `rules.md §5`).

Exit criteria: `docker compose up` brings up all services cleanly with health checks passing.

Phase 1 — Data Modeling & Ingestion (Sprint 1)

Goal: Prove data can be reliably acquired and staged.

Finalize the PostgreSQL schema (`design.md §2`) via migrations.

Implement the `staging_jobs` pipeline.

Validate ingestion against **at least one API-based source and one scraper-based source**.

Implement token-bucket rate limiting and exponential backoff with jitter per source.

Exit criteria: Raw postings from 2+ sources land in `staging_jobs` with correct timestamps and source attribution; malformed records are quarantined, not dropped or crashing the worker.

Phase 2 — Standardization & Deduplication (Sprint 2)

Goal: Prove the same job from two platforms becomes one canonical record.

Implement field mapping per source.

Implement text normalization (case-folding, legal-suffix stripping) and location standardization.

Implement the composite deduplication heuristic and `dedup_hash` fast path (`design.md §4.1`).

Build a manually labeled test set of known duplicate/non-duplicate posting pairs; validate heuristic thresholds against it.

Exit criteria: Deduplication precision/recall against the labeled test set meets an agreed target (define numeric target before sprint close); canonical `jobs` table populated from ≥3 sources with no unresolved duplicates in the test set.

Phase 3 — NLP Skill Extraction (Sprint 3)

Goal: Prove skills can be reliably extracted from unstructured text.

Build/curate the skill-taxonomy gazetteer.

Train and integrate the spaCy custom NER pipeline (Phase A, `design.md §4.2`).

Manually annotate a sample of job descriptions; measure extraction precision and recall against it.

Persist scored results to `job_skills` .

Exit criteria: Extraction precision/recall against the annotated sample meets an agreed target; `relevance_score` distribution is sane (not clustered at 0 or 1 uniformly).

Phase 4 — Matching Engine & API (Sprint 4)

Goal: Prove a user's skill profile can be scored against the job corpus.

Implement TF-IDF vectorization + cosine similarity (Phase B, `design.md §4.3`).

Expose `GET /api/v1/feed` through FastAPI, including skill-overlap annotations.

Implement remaining v1 endpoints (`design.md §3.2`): auth, profile, skills, applications.

Add caching for pre-computable match scores.

Exit criteria: A test user with a known skill profile receives a feed ranked in a way that's manually verifiable as sensible (higher-overlap jobs rank above lower-overlap jobs).

Phase 5 — Frontend Integration (Sprint 5)

Goal: Prove the whole pipeline is usable end-to-end by a human.

Build the React/Next.js client against the finalized API.

Implement skill profile management UI.

Implement the ranked feed view with match-explanation UI (overlapping/missing skills).

Implement application tracking UI (`SAVED` → `APPLIED` → ...).

Exit criteria: A new user can register, declare skills, browse a ranked feed, and track an application without touching the API directly.

Phase 6 — Integration Testing & Refinement (Sprint 6)

Goal: Validate the full pipeline against live, multi-source data and tune it.

End-to-end test against live ingestion from all configured sources.

Iteratively tune deduplication thresholds and match-scoring weights based on observed output quality.

Load-test the feed endpoint; validate caching effectiveness.

Security pass: auth flows, input validation, rate-limit enforcement.

Exit criteria: All four PoC success criteria from the synopsis abstract are demonstrable:

Automated, deduplicated aggregation from ≥3 sources.

Measurable-precision NLP skill extraction.

Quantifiable semantic match score per user/job pair.

Ranked, personalized feed delivered through the web UI.

Phase 7 — Hardening & Deployment (post-PoC)

Goal: Move from PoC to a deployable staging/production system.

Production Docker Compose / infra sizing per `Architecture.md` §5.

Observability: structured logging, worker task monitoring, error alerting.

Backup/restore procedure for PostgreSQL.

Documentation pass on all API endpoints (OpenAPI is generated, but usage docs still need writing).

Phase 8+ — Future Enhancements (Roadmap, not committed scope)

These are explicitly **out of scope for the initial PoC** but are architecturally anticipated:

AI-driven resume parsing: extend the Phase A NLP pipeline to accept a PDF resume, extract text, and auto-populate `user_skills` (including inferred `years_of_experience` from parsed employment dates).

Automated daily match alerts: a scheduled Celery task re-running Phase B against newly ingested postings, emailing users whose `match_score` crosses a configurable high-confidence threshold (e.g. $\geq 90\%$).

Expansion beyond the 3-source minimum via the existing modular ingestion architecture.

Migration from TF-IDF to dense transformer embeddings.

User feedback signals ("not relevant" flags) as a supervised fine-tuning signal.

Employer-facing analytics dashboard on aggregate, anonymized skill-demand trends.

Phase-to-Document Cross-Reference

Phase	Primary reference doc(s)
0	<code>Architecture.md</code> §5 (deployment), <code>rules.md</code>
1	<code>Architecture.md</code> §3.1, <code>rules.md</code> §1 (source compliance)
2	<code>design.md</code> §4.1, <code>design.md</code> §2 (schema)
3	<code>design.md</code> §4.2
4	<code>design.md</code> §4.3, <code>design.md</code> §3 (API)
5	<code>design.md</code> §5 (UX)
6	All docs — this is the integration checkpoint
8+	<code>memory.md</code> (tracks roadmap decisions as they're taken up)

Companion documents: `Architecture.md` (system structure), `design.md` (schema/API/algorithms), `rules.md` (engineering rules), `memory.md` (decision log).